

HABIT TRACKER

A PROJECT REPORT

Submitted by

HARISH R P 2116230701106

in partial fulfillment for the course

CS23621 – MOBILE APPLICATION DEVELOPMENT LABORATORY

of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR

THANDALAM

CHENNAI – 602 105

MAY 2026

RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI - 602105

BONAFIDE CERTIFICATE

Certified that this project report **HABIT TRACKER** is the bonafide work of **HARISH R P (230701106)** who carried out the project work (CS23621-Mobile Application Development Laboratory) under my supervision.

Mr.B.Bhuvaneswaran
SUPERVISOR
Assistant Professor (SG)
Department of
Computer Science and Engineering
Rajalakshmi Engineering College
Rajalakshmi Nagar
Thandalam
Chennai - 602105

Submitted to Project viva-voce examination held on _____.

Internal Examiner

External Examiner

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	1
1.	INTRODUCTION	2
	1.1 GENERAL	2
	1.2 OBJECTIVE	2
	1.3 EXISTING SYSTEM	3
	1.4 PROPOSED SYSTEM	4
2.	LITERATURE REVIEW	5
	2.1 GENERAL	5
3.	SYSTEM DESIGN	7
	3.1 GENERAL	8
	3.1.1 SYSTEM FLOW DIAGRAM	9
	3.1.2 ARCHITECTURE DIAGRAM	10
	3.1.3 USE CASE DIAGRAM	10
4.	PROJECT DESCRIPTION	11
	4.1 METHODOLOGIE	11
	4.1.1 MODULES	12
5.	CONCLUSIONS	13
	5.1 GENERAL	13
	REFERENCES	14
	APPENDICES	16

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S.Meganathan, B.E, F.I.E.**, our Vice Chairman **Mr. Abhay Meganathan, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) Thangam Meganathan, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N.Murugesan, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. E. M. Malathy, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mr. B.Bhuvaneswaran, M.E.**, Assistant Professor (SG), Department of Computer Science and Engineering, Rajalakshmi Engineering College for their valuable guidance throughout the course of the project. We are very glad to thank our Project Coordinator, **Mr. B.Bhuvaneswaran, M.E.**, Assistant Professor (SG), Department of Computer Science and Engineering for his useful tips during our review to build our project.

Student Name (Register Number)

ABSTRACT

HabitBuilder is an Android-based habit tracking and productivity management application developed to help users create, monitor, and maintain positive daily habits effectively. In today's fast-paced digital environment, many individuals face difficulty in maintaining consistency in activities such as exercising, studying, reading, meditation, and time management. HabitBuilder provides a simple and organized solution for users to track their routines and improve productivity.

The application is developed using Kotlin and modern Android development technologies such as Jetpack components, Room Database, Lifecycle libraries, ViewModel, and modular architecture principles. The project follows a multi-module structure that separates core functionalities, database management, UI components, utilities, and feature modules. This modular approach improves maintainability, scalability, testing capability, and code reusability.

HabitBuilder allows users to create and schedule habits, monitor completion status, maintain streak records, and analyze productivity trends through simple analytics and dashboards. The application stores user data efficiently using Room Database, enabling reliable offline access and smooth performance.

The project demonstrates the implementation of modern Android software engineering practices while providing a practical productivity solution for everyday users. Future improvements may include cloud synchronization, AI-based habit recommendations, wearable integration, and real-time notifications.

1.INTRODUCTION

1.1 GENERAL

Technology has become an important part of everyday life, especially in improving productivity and managing personal activities. Many people struggle to maintain consistency in their daily routines due to busy schedules, lack of motivation, and poor tracking mechanisms. Habit tracking applications help users monitor their progress and encourage discipline through continuous tracking and reminders.

HabitBuilder is a mobile application developed for Android devices that focuses on helping users create and maintain productive habits. The application provides an organized environment where users can add habits, schedule activities, mark daily completion, and monitor their progress through analytics and streak systems. The application promotes self-discipline and encourages users to maintain healthy routines consistently.

The project is developed using a modular multi-module architecture where different functionalities are separated into independent modules such as database management, UI handling, testing utilities, and feature modules. This approach improves code maintainability, reduces dependency conflicts, and simplifies future feature integration.

HabitBuilder also demonstrates modern Android development techniques using Kotlin programming language and Android Jetpack libraries. The project follows software engineering principles such as modularization, reusability, separation of concerns, and test-driven development.

1.2 OBJECTIVE

The primary objective of HabitBuilder is to provide users with a reliable and efficient platform for tracking daily habits and improving productivity. The application is designed to make habit management simple, interactive, and motivating.

The main objectives are:

- To develop a user-friendly Android application for habit management.
- To help users build positive daily routines through continuous tracking.
- To maintain habit streaks and completion statistics.
- To provide productivity insights using analytics and dashboards.

- To implement modern Android development practices using Kotlin.
- To design the application using modular architecture for better scalability.
- To enable reusable UI and utility components across different modules.
- To improve maintainability and testing efficiency through modularization.
- To provide efficient local data storage using Room Database.
- To support future enhancements and feature expansion.

The system aims to improve both user productivity and software development efficiency.

1.3 EXISTING SYSTEM

Several habit tracking applications already exist in the market. These applications provide basic functionalities such as task reminders, daily checklists, notifications, and productivity tracking. However, many existing systems suffer from technical and usability limitations.

Most traditional habit tracking applications are developed using monolithic architectures where all functionalities are tightly connected in a single module. This increases code complexity and makes future maintenance difficult. Some applications also provide overly complex interfaces that reduce user engagement and usability.

Existing systems may also have issues such as:

- Lack of modular design.
- Poor scalability for adding new features.
- Limited offline functionality.
- Dependency on internet connectivity.
- Difficult testing and debugging processes.
- Limited customization options.
- Performance issues on low-end devices.

In many systems, the user experience becomes less effective because of unnecessary features and poor data organization. Additionally, some

applications do not provide meaningful analytics or visual tracking mechanisms to motivate users.

Disadvantages of Existing System

- Difficult to maintain large codebases.
- Increased application complexity.
- Reduced code reusability.
- Limited scalability.
- Poor modular separation.
- Low testing efficiency.
- Less optimized performance.
- Inconsistent UI design.

These limitations motivated the development of HabitBuilder using a cleaner and modular architecture.

1.4 PROPOSED SYSTEM

The proposed system, HabitBuilder, is designed to overcome the limitations of existing habit tracking applications by implementing a scalable and modular Android architecture. The application provides a structured environment for users to create habits, monitor daily progress, and analyze productivity trends.

HabitBuilder separates functionalities into reusable modules such as core models, database handling, UI components, common utilities, and feature modules. This modular architecture improves code organization and simplifies application maintenance.

The system includes features such as:

- Habit creation and editing.
- Daily habit completion tracking.
- Streak monitoring and analytics.
- Dashboard-based progress visualization.
- Shared UI components.
- Local offline database storage.

- Modular feature implementation.

The application uses Room Database for efficient local data management and Kotlin programming language for modern Android development. The use of Android Jetpack components improves application lifecycle handling and enhances performance.

Advantages of Proposed System

- Better scalability and maintainability.
- Improved code organization.
- Reusable components and utilities.
- Easier testing and debugging.
- Faster feature integration.
- User-friendly interface.
- Efficient local storage support.
- Reduced code duplication.

Applications

HabitBuilder can be used in various real-life productivity scenarios such as:

- Personal productivity management.
- Student study planning.
- Fitness and workout tracking.
- Reading and learning schedules.
- Time management systems.
- Daily routine monitoring.

The proposed system provides both technical efficiency and practical usability for users.

2.LITERATURE REVIEW

2.1 GENERAL

Habit tracking systems have gained popularity in recent years because of increasing awareness regarding productivity, mental wellness, and self-improvement. Research studies show that tracking habits regularly improves behavioral consistency and increases the chances of achieving long-term goals.

Several mobile productivity applications use reminders, notifications, streak systems, and analytics to encourage user engagement. These systems are designed based on behavioral psychology concepts where repeated actions over time form consistent habits.

Modern Android development has also evolved significantly with the introduction of Kotlin and Jetpack libraries. Kotlin provides concise syntax, null safety, and interoperability with Java, making Android application development faster and safer.

Android Jetpack libraries simplify many complex tasks such as lifecycle management, database operations, UI rendering, and dependency management.

Review of Related Technologies

Kotlin

Kotlin is a modern programming language officially supported by Google for Android development. It provides improved readability, reduced boilerplate code, null safety features, and seamless interoperability with Java.

Room Database

Room Database is an abstraction layer over SQLite that simplifies database management using entities, DAOs, and queries. It improves database reliability and reduces SQL-related errors.

MVVM Architecture

MVVM architecture separates the application into Model, View, and ViewModel layers. This separation improves maintainability, testability, and code organization.

Jetpack Compose

Jetpack Compose is Android's modern UI toolkit for building native user interfaces using declarative programming methods. It reduces UI complexity and improves development speed.

Existing Research Findings

Research studies indicate that:

- Consistent habit tracking improves self-discipline.
- Visual analytics increase user motivation.
- Reminder systems improve habit completion rates.
- Simple user interfaces improve engagement.
- Modular software architecture improves application scalability.

The proposed HabitBuilder system integrates these research concepts into a modern Android application.

3.SYSTEM DESIGN

3.1 GENERAL

System design describes the internal structure and interaction between different components of the application. HabitBuilder follows a modular MVVM architecture that separates business logic, user interface, and data management into independent modules.

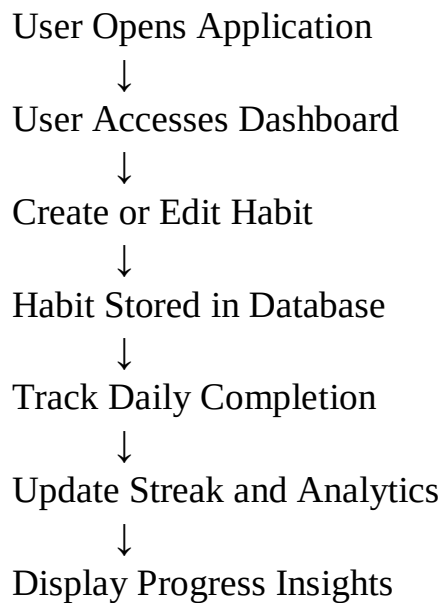
The system consists of several layers including:

- User Interface Layer
- Business Logic Layer
- Data Layer
- Database Layer
- Feature Modules
- Shared Core Modules

This separation improves maintainability, reduces complexity, and enables efficient testing.

3.1.1 SYSTEM FLOW DIAGRAM

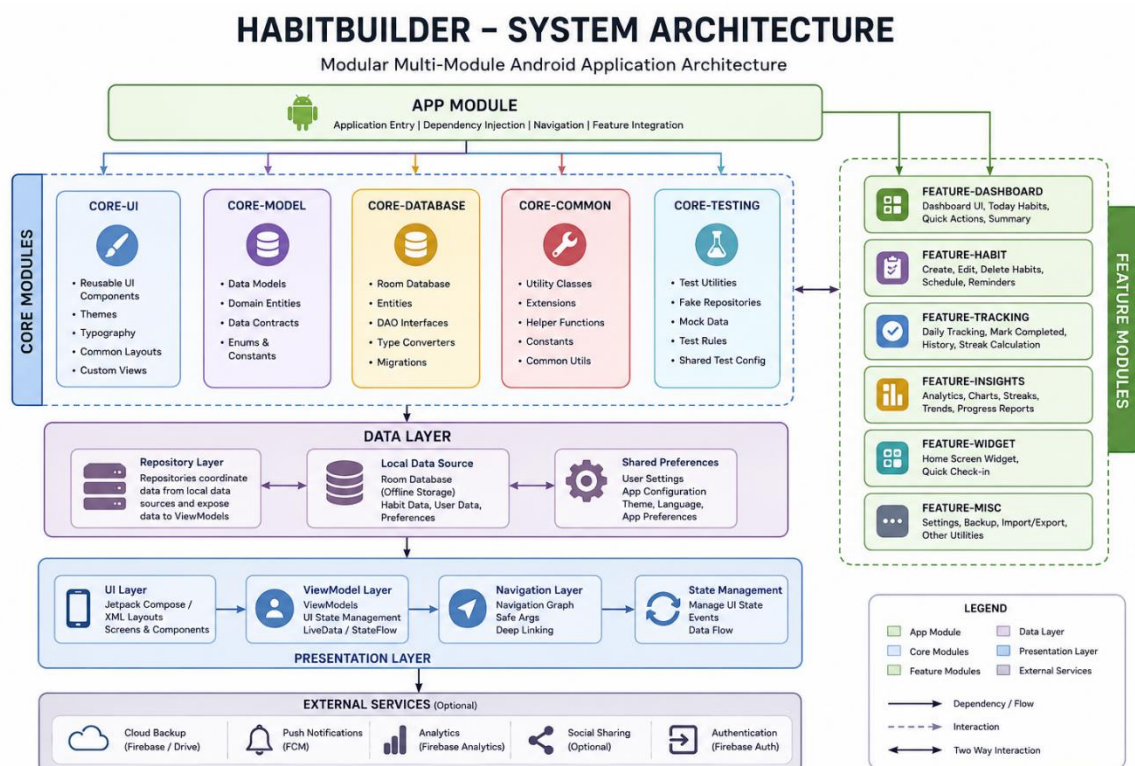
System Flow



Explanation

The user first opens the application and accesses the dashboard interface. Users can create new habits or edit existing habits according to their requirements. The habit information is stored locally using Room Database. Users can then mark habits as completed daily, after which the system updates streak records and analytics automatically. Finally, productivity insights and progress reports are displayed to the user.

3.1.2 ARCHITECTURE DIAGRAM



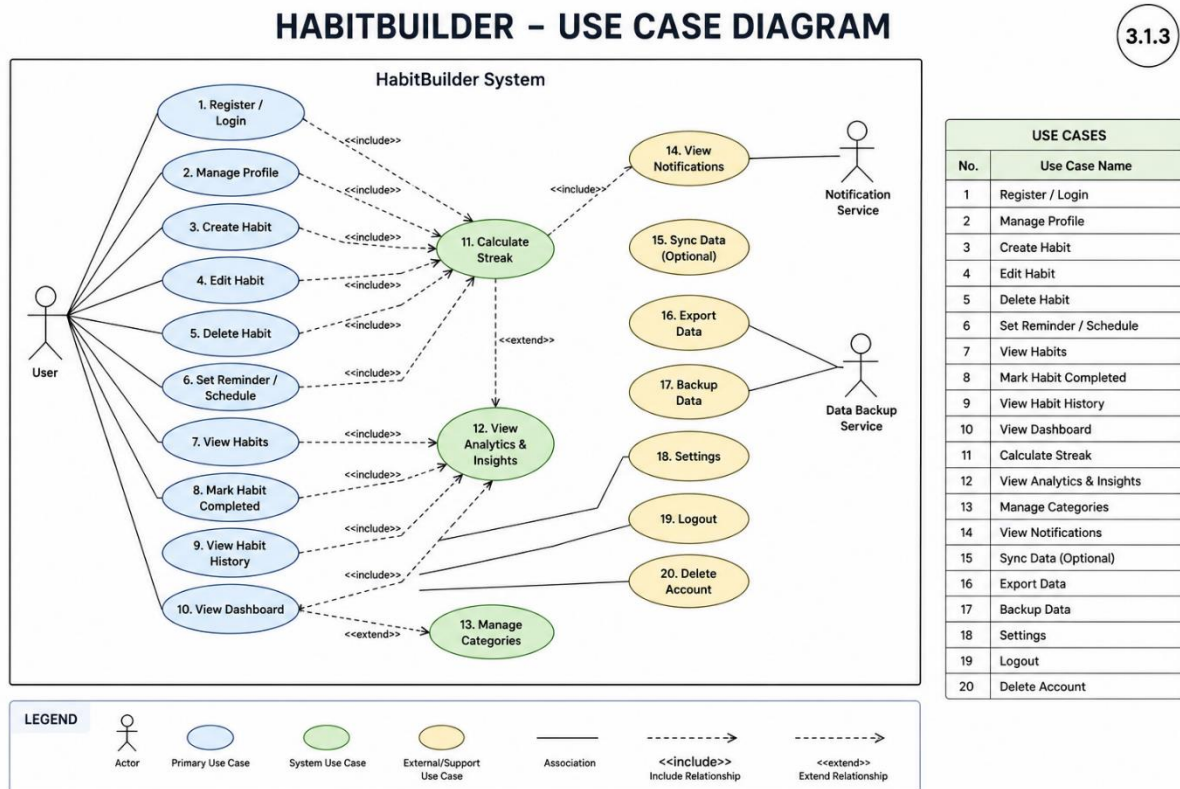
Explanation

The architecture contains a central application module connected to multiple reusable core modules and feature modules.

- App Module handles navigation and application initialization.
- Core Model Module stores entities and domain models.
- Core Database Module handles Room Database operations.

- Core UI Module contains reusable UI components and themes.
- Core Common Module contains utility functions and helpers.

3.1.3 USE CASE DIAGRAM



Description

The user interacts with the system to perform several operations such as creating habits, editing schedules, tracking completion status, and viewing productivity analytics. The application processes user actions and updates the database accordingly.

The use case diagram illustrates the relationship between the user and the major functionalities of the system.

PROJECT DESCRIPTION

4.1 METHODOLOGIE

The development of HabitBuilder follows an Agile-based software development methodology. The project is implemented in stages including requirement analysis, system design, coding, testing, and deployment.

The methodology focuses on modular development where functionalities are divided into independent modules. Each module is tested individually before integration into the complete system.

Development Process

1. Requirement Gathering
2. System Analysis
3. UI Design
4. Database Design
5. Module Development
6. Integration Testing
7. Deployment and Maintenance

This methodology improves development efficiency and simplifies debugging.

4.1.1 MODULES

1. App Module

The App module serves as the main entry point of the application. It manages application initialization, navigation, dependency injection, and communication between feature modules.

2. Core Model Module

This module contains data classes, entities, and domain models used throughout the application. It acts as a centralized layer for handling application data structures.

3. Core Database Module

The Core Database module manages Room Database configurations, DAO interfaces, queries, and database transactions. It ensures reliable local data storage and retrieval.

4. Core UI Module

This module contains reusable UI components, themes, buttons, layouts, typography styles, and shared interface utilities that maintain consistent application design.

5. Core Common Module

The Common module provides utility classes, helper methods, constants, extensions, and shared functions used across different modules.

6. Core Testing Module

This module contains testing utilities such as fake repositories, mock data generators, and shared testing configurations to simplify unit testing and integration testing.

7. Feature Modules

Feature modules contain independent functionalities such as:

- Dashboard Module
- Habit Tracking Module
- Analytics Module
- Widget Module

This separation improves modularity and simplifies future feature additions.

Software Requirements

Software	Specification
Operating System	Windows/Linux/macOS
IDE	Android Studio
Language	Kotlin
SDK	Android SDK
Database	Room Database
Build Tool	Gradle

Explanation

The project uses Android Studio as the development environment along with Kotlin programming language and Android SDK tools. Gradle is used for dependency management and application building.

Hardware Requirements

Hardware	Specification
Processor	Intel i3 or above
RAM	8 GB minimum
Storage	10 GB free space
Mobile Device	Android Smartphone

Explanation

The system requires a moderate hardware configuration for development and testing. Android devices or emulators are used for running and validating the application.

Features and Functionalities

- Create and manage habits.
- Daily completion tracking.
- Habit streak calculation.
- Productivity analytics.
- Dashboard visualization.
- Local database storage.
- Reusable UI system.
- Modular architecture support.
- Unit and instrumentation testing.

Habit Tracking Workflow

Create Habit



Store in Database



Daily Completion Update



Calculate Streak



Display Analytics

Testing

Testing is performed to ensure application reliability and performance.

Unit Testing

Used for testing business logic, repositories, ViewModels, and utility functions.

Instrumentation Testing

Used for testing UI behavior and application interaction on Android devices.

Integration Testing

Used for validating communication between modules and database operations.

Testing improves software quality and reduces system failures.

CONCLUSIONS

5.1 GENERAL

HabitBuilder is a modern Android application designed to help users build positive habits and improve productivity through continuous monitoring and tracking mechanisms. The project successfully demonstrates the implementation of modular architecture, modern Android development tools, and software engineering principles.

The application provides an organized and user-friendly environment for managing habits, monitoring streaks, and analyzing productivity trends. The use of Kotlin, Room Database, Jetpack libraries, and modular architecture improves maintainability, scalability, and testing efficiency.

The project also highlights the importance of reusable components and separation of concerns in large-scale Android application development. HabitBuilder can be extended further with advanced features such as cloud synchronization, AI-based recommendations, wearable support, and push notifications.

Overall, the project achieves its objectives by combining practical productivity features with efficient software architecture practices.

REFERENCES

1. Android Developers Documentation – Android Architecture Components.
2. Kotlin Official Documentation.
3. Google Android Jetpack Documentation.
4. Room Persistence Library Documentation.
5. MVVM Architecture Design Patterns for Android Applications.
6. Research Papers on Habit Tracking and Behavioral Analysis.
7. Android Studio Development Guidelines.
8. Software Engineering Principles for Modular Systems.

APPENDICES

Appendix A – Sample Gradle Command

```
gradlew.bat assembleDebug  
gradlew.bat test
```

Appendix B – Sample Kotlin Code

```
@Entity(tableName = "habits")  
data class Habit(  
    @PrimaryKey(autoGenerate = true)  
    val id: Int = 0,  
    val title: String,  
    val completed: Boolean  
)
```

Appendix C – Future Enhancements

- Firebase Cloud Synchronization
- AI-based Habit Suggestions
- Push Notification System
- Wearable Device Integration
- Dark Mode Enhancements
- Cross-platform Support
- Backup and Restore Features